Text Summarization

using Bi-Directional LSTM + Attention Mechanism and BART

Dataset:

This project uses ***CNN/DailyMail*** dataset ***(version 3.0.0)*** from *Hugging Face Datasets Library*. This version of dataset is a fully-anonymized version which means it does NOT have the id column (retains original names and does not give entity) and only has Article and Highlights, making it suitable for real-world NLP applications.It comes with standardized train/test/validation splits.

Rationale behind using hugging face library:

- Manual loading of cnn_dailymail CSV dataset resulted in extreme data loss. only 4775 samples were loading with a split of (1414,1732,1629) for training, validation and testing. There were parsing issues which was leading to skipped rows.
- Using Hugging Face Datasets library: It becomes easy to access the dataset for others to reproduce. Also the library handles chaching and downloading automatically, making it efficient.

Link: https://huggingface.co/datasets/abisee/cnn_dailymail

The link above is for the general version of the cnn_dailymail dataset. Here, I am accessing the version 3.0.0 of the dataset.

Dataset Structure:

- Articles - news articles from CNN and Daily Mail
- Highlights - human written short summaries (abstractive)

**Dataset Statistics of version 3.0.0 are: **

- Volume - around 300,000 unique article + summary pairs
- Artice token count - 781 words
- Summary token count - 56 words
- Training samples - 287,113
- Validation samples - 13,368
- Test samples - 11,490

Approach for Text Summarization:

1. Data Preparation

- Load the libraries and the dataset

- Pre-processing the text (cleaning, handling missing values, tokenising, padding)
- Creating decoder inputs with start tokens

2. Defining the Model Architecture

- buliding seq2seq model with bi-directional LSTM and Attention mechanism
- Dropout for regularization
- Compiling model with Adam optimizer and sparse categorical crossentropy loss

3. Training and Fine-tuning

- Setting early stopping & model checkpoint
- Training the model
- Evaluating and visualizing training metrics

Data Loading / Acquisition

The Libraries used along with their purpose:

- pd - pandas for data manipulation
- np - numpy for mathematical operations
- re - regular expression (regex) for string manipulation
- nltk - natural language toolkit for analysing textual data
- stopwords - for removing the stopwords for pre-processing (the, and, is, etc)
- tokenize - importing word_tokenizer to break text into individual tokens
- WordNetLemmetizer - for bringing words to thier base form
- tf - tensorflow library
- Tokenizer - class of tf API, for converting text to sequences
- pad_sequences - for padding them 0 for same length
- plt - Matplotlib for visualization
- pickle - for saving the models
- os - for handling file paths and directories

```
#importing libraries
import pandas as pd
import numpy as np
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize as tokenize
from nltk.stem import WordNetLemmatizer
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import matplotlib.pyplot as plt
import pickle
```

```
import os

#download nltk resources
nltk.download('punkt') # for tokenizing text into words/sentences
nltk.download('stopwords') #list of stopwords to remove
nltk.download('wordnet') #base form of words

np.random.seed(42) #for reproducibility
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
```

```
#installing the latest version of the datasets library
!pip install --upgrade datasets
```

```
Collecting datasets
  Downloading datasets-3.5.0-py3-none-any.whl.metadata (19 kB)
Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-pa
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.11/dist
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.11/
Collecting dill<0.3.9,>=0.3.0 (from datasets)
  Downloading dill-0.3.8-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-pacl
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.1
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.11/dis
Collecting xxhash (from datasets)
  Downloading xxhash-3.5.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_
Collecting multiprocess<0.70.17 (from datasets)
  Downloading multiprocess-0.70.16-py311-none-any.whl.metadata (7.2 kB)
Collecting fsspec<=2024.12.0,>=2023.1.0 (from fsspec[http]<=2024.12.0,>=202
  Downloading fsspec-2024.12.0-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: aiohttp in /usr/local/lib/python3.11/dist-pac
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/pyt
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-p
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.11/dist
Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/pyt
Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.1
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.11/d
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.1
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/py
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dis
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/pyth
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dis
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-pa
Downloading datasets-3.5.0-py3-none-any.whl (491 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 491.2/491.2 kB 15.9 MB/s eta 0:
Downloading dill-0.3.8-py3-none-any.whl (116 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 116.3/116.3 kB 11.7 MB/s eta 0:
```

```
Downloading fsspec-2024.12.0-py3-none-any.whl (183 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 183.9/183.9 kB 17.7 MB/s eta 0:(
Downloading multiprocess-0.70.16-py311-none-any.whl (143 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 143.5/143.5 kB 13.1 MB/s eta 0:(
Downloading xxhash-3.5.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x8(
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 194.8/194.8 kB 18.7 MB/s eta 0:(
Installing collected packages: xxhash, fsspec, dill, multiprocess, datasets
  Attempting uninstall: fsspec
    Found existing installation: fsspec 2025.3.2
    Uninstalling fsspec-2025.3.2:
      Successfully uninstalled fsspec-2025.3.2
ERROR: pip's dependency resolver does not currently take into account all tl
torch 2.6.0+cu124 requires nvidia-cublas-cu12==12.4.5.8; platform_system ==
torch 2.6.0+cu124 requires nvidia-cuda-cupti-cu12==12.4.127; platform_syster
torch 2.6.0+cu124 requires nvidia-cuda-nvrtc-cu12==12.4.127; platform_syster
torch 2.6.0+cu124 requires nvidia-cuda-runtime-cu12==12.4.127; platform_sys
torch 2.6.0+cu124 requires nvidia-cudnn-cu12==9.1.0.70; platform_system ==
```

```
#importing and loading the dataset of version 3.0.0 from Hugging Face librar
from datasets import load_dataset
dataset = load_dataset("cnn_dailymail", "3.0.0")
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: Us
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings ta
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access p
  warnings.warn(
README.md: 100%                                    15.6k/15.6k [00:00<00:00, 486kB/s]

train-00000-of-                                    257M/257M [00:03<00:00, 88.2MB/s]

00003.parquet: 100%

train-00001-of-                                    257M/257M [00:02<00:00, 96.9MB/s]

00003.parquet: 100%

train-00002-of-                                    259M/259M [00:02<00:00, 141MB/s]

00003.parquet: 100%

validation-00000-of-                               34.7M/34.7M [00:00<00:00, 111MB/s]

00001.parquet: 100%

test-00000-of-                                     30.0M/30.0M [00:00<00:00, 117MB/s]
```

```
# Convert to pandas DataFrame
train_df = pd.DataFrame({
    'article': dataset['train']['article'],
    'highlights': dataset['train']['highlights']
})

val_df = pd.DataFrame({
    'article': dataset['validation']['article'],
    'highlights': dataset['validation']['highlights']
})
```

```
test_df = pd.DataFrame({
    'article': dataset['test']['article'],
    'highlights': dataset['test']['highlights']
})

# Check dataset info
print(f"Training set shape: {train_df.shape}")
print(f"Validation set shape: {val_df.shape}")
print(f"Test set shape: {test_df.shape}")
```

```
Training set shape: (287113, 2)
Validation set shape: (13368, 2)
Test set shape: (11490, 2)
```

```
#checking missing values
print('Missing values in training dataset:\n')
print(train_df.isnull().sum())
```

```
Missing values in training dataset:

article       0
highlights    0
dtype: int64
```

Data Cleaning & Pre-Processing

The below code performs text cleaning and pre-processing tasks.

The cleaning function involves removing the HTML tags, any special characters and stopwords, converting to lower case, tokenizing and lemmetizing.

Due to computational constraint and time-sensitive nature of the project, I have used sub-sample of the dataset for faster training for all the 3 sets by using sample() method. The samples are set to 1000 for training and 500 for validation and 200 for test sets.

Now, just to be sure, I check for any missing values in the cleaned datasets and drop if any.

```
#Text cleaning
#reference for writing: from lab sheet (lab NLP)
import nltk
nltk.download('punkt_tab')
stop_words = set(stopwords.words('english'))
#lemmatizer = WordNetLemmatizer()

def text_cleaned(text):
  """
  cleaning and pre-processing text by removing/handling html tags, special c
```

```
    lower case and lemmetization to bring all the words to it's root word.
    """
    lemmatizer = WordNetLemmatizer()

    if isinstance(text, str):
      text = re.sub(r'<.*?>', '', text)  #removing any HTML tags <>
      text = re.sub(r'[^A-Za-z0-9\s]', ' ', text) #removing the special charac
      text = text.lower() #lower-casing the text
      words = tokenize(text) #breaking the text into individual tokens

      words = [lemmatizer.lemmatize(word) for word in words if word not in sto
      return ' '.join(words)
    else:
      return ''

#As the dataset is too large, I am using a subset / small part of dataset fo
train_sample = 1000
val_sample = 500
test_sample = 200

#sample() here randomly selects a subset of the dataset, but not crossing th
#setting random_state ensures that, the code chooses the same subset everyti
train_df = train_df.sample(min(train_sample, len(train_df)), random_state=42
val_df = val_df.sample(min(val_sample, len(val_df)), random_state=42)
test_df = test_df.sample(min(test_sample, len(test_df)), random_state=42)

print(f"Training set shape after sampling: {train_df.shape}")
print(f"Validation set shape after sampling: {val_df.shape}")
print(f"Test set shape after sampling: {test_df.shape}")

#applying the text_cleaned function on the datasets.
train_df['clean_article'] = train_df['article'].apply(text_cleaned)
train_df['clean_highlights'] = train_df['highlights'].apply(text_cleaned)
val_df['clean_article'] = val_df['article'].apply(text_cleaned)
val_df['clean_highlights'] = val_df['highlights'].apply(text_cleaned)
test_df['clean_article'] = test_df['article'].apply(text_cleaned)
test_df['clean_highlights'] = test_df['highlights'].apply(text_cleaned)

#if there are any missing values in any dataset, I want to remove them befor
train_df.dropna(subset=['clean_article', 'clean_highlights'], inplace=True)
val_df.dropna(subset=['clean_article', 'clean_highlights'], inplace=True)
test_df.dropna(subset=['clean_article', 'clean_highlights'], inplace=True)

print(f"Training set shape after cleaning: {train_df.shape}")
print(f"Validation set shape after cleaning: {val_df.shape}")
print(f"Test set shape after cleaning: {test_df.shape}")
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Training set shape after sampling: (1000, 2)
Validation set shape after sampling: (500, 2)
Test set shape after sampling: (200, 2)
```

```
Training set shape after cleaning: (1000, 4)
Validation set shape after cleaning: (500, 4)
Test set shape after cleaning: (200, 4)
```

```
print(train_df.columns)
```

```
Index(['article', 'highlights', 'clean_article', 'clean_highlights'], dtype='
```

```
#assigning the parameters before training
max_article = 200  #max words allowed in article input
max_highlight = 40 #max words allowed in highlights/summary
vocab_size = 5000 #size of the vocabulary for tokenizer
embedding_dim = 32 #dimension of the word embedding for vector representatio
LSTM_units = 32 #number of neurons in the layer
batch_size = 16  #number of samples trained before weight changes
epochs = 10 #number of iterations
```

The next step will prepare the data for the model by tokenizing, converting text to numerical sequences and padding the sequences.

1. Two different tokenizers are created for two features and are given the limit of the vocab_size. The words that lie outside of this size are replaced with Out-of-vocabulary tokens (OOV).
2. Each text is converted into sequence of numbers, because padding can only be applied on numerical data and not on words or text.
3. Sequences are standardized to fixed lengths, specified in the code performed above. Shorter sequences will be padded with 0 in the end whereas the longer ones will be cut at the end.

```
#tokenizing and padding articles and summaries
tokenizer_article = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer_article.fit_on_texts(list(train_df['clean_article']) + list(val_df
tokenizer_highlight = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer_highlight.fit_on_texts(list(train_df['clean_highlights']) + list(v

#converting articles to sequences before padding
train_article_to_seq = tokenizer_article.texts_to_sequences(train_df['clean_
val_article_to_seq = tokenizer_article.texts_to_sequences(val_df['clean_arti
test_article_to_seq = tokenizer_article.texts_to_sequences(test_df['clean_ar

#converting highlights/summaries to sequences
#where each token will be given a unique numerical id
train_highlight_to_seq = tokenizer_highlight.texts_to_sequences(train_df['cl
val_highlight_to_seq = tokenizer_highlight.texts_to_sequences(val_df['clean_
test_highlight_to_seq = tokenizer_highlight.texts_to_sequences(test_df['clea

#padding the article sequences
train_article_pad = pad_sequences(train_article_to_seq, maxlen=max_article,
```

```
val_article_pad = pad_sequences(val_article_to_seq, maxlen=max_article, padd
test_article_pad = pad_sequences(test_article_to_seq, maxlen=max_article, pa

#padding the highlights sequences
train_highlight_pad = pad_sequences(train_highlight_to_seq, maxlen=max_highl
val_highlight_pad = pad_sequences(val_highlight_to_seq, maxlen=max_highlight
test_highlight_pad = pad_sequences(test_highlight_to_seq, maxlen=max_highlig

print(f"Training set shape after tokenization and padding: {train_article_pa
print(f"Validation set shape after tokenization and padding: {val_article_pa
print(f"Test set shape after tokenization and padding: {test_article_pad.sha
```

```
Training set shape after tokenization and padding: (1000, 200)
Validation set shape after tokenization and padding: (500, 200)
Test set shape after tokenization and padding: (200, 200)
```

```
print(train_highlight_pad.shape)
print(val_highlight_pad.shape)
print(test_highlight_pad.shape)
```

```
(1000, 40)
(500, 40)
(200, 40)
```

Defining Model Architecture

This is a ***Bi-LSTM based seq2seq, encoder + decoder model architecture with attention***.

Bi-directional LSTMs is used as they are proficient in handling long-text well and understand the context of the text more accurately by captueing most of the semantics from both the sides.

The encoder Bi-LSTM takes the article as input and produces hidden states for each and every single input token. The decoder LSTM will generate summary one token at a time. I am applying a shifting operation which creates a '*teacher-forcing*' approach. In this approach, the decoder's first input is a *start* token. It ensures that, during training, each predicted token becomes the input for the next steps.

The ***Attention*** will put more weight on more relevant/important words of the encoder output.

Here, the decoder input will be shifted right by 1 position with a start token for both training and validation sets. Decoder output will have the original summary sequence.

```
#creating decoder inputs for training
train_decoder_input = np.zeros_like(train_highlight_pad) #creating the array
```

```
train_decoder_input[:, 1:] = train_highlight_pad[:, :-1] #shifting one posit
train_decoder_input[:, 0] = 1  #start token

val_decoder_input = np.zeros_like(val_highlight_pad)
val_decoder_input[:, 1:] = val_highlight_pad[:, :-1]
val_decoder_input[:, 0] = 1  #start token

test_decoder_input = np.zeros_like(test_highlight_pad)
test_decoder_input[:, 1:] = test_highlight_pad[:, :-1]
test_decoder_input[:, 0] = 1  #start token

print('Train Decoder Input Shape: ',train_decoder_input.shape)
print('Val Decoder Input Shape: ',val_decoder_input.shape)
print('Test Decoder Input Shape: ',test_decoder_input.shape)

#assigning vocabulary sizes(counts the number of unique tokens and adds 1 fo
'''
tokenizer_article.word_index will map words to their index values which star
thus we add 1 to get the correct number of unique tokens.
'''
article_vocab_size = min(len(tokenizer_article.word_index) + 1, vocab_size)
highlight_vocab_size = min(len(tokenizer_highlight.word_index) + 1, vocab_si

print(f"Article Vocabulary Size: {article_vocab_size}")
print(f"Highlight Vocabulary Size: {highlight_vocab_size}")
```

```
Train Decoder Input Shape:  (1000, 40)
Val Decoder Input Shape:  (500, 40)
Test Decoder Input Shape:  (200, 40)
Article Vocabulary Size: 5000
Highlight Vocabulary Size: 5000
```

Model Definition

- importing necessary tf libraries
- Defining the **seq2seq model function**
- Encoder - consisting of input layer, embedding layer, bidirectional LSTM, combines both the hidden states
- Decoder - consisting of decoder inputs(shifted), embedding layer, LSTM layer
- Attention Mechanism - consisting of attention layer, context vector (weighted sum) and combines decoder output with context vector for comprehensive information.
- Output layer - Dense layer with **softmax** activation and **dropout** to prevent overfitting.
- Compiling model - consisting of **Adam Optimizer**, **Sparse categorical crossentropy as loss function** and **Accuracy for metrics**.
- Building model - creating the model instance and printing the model summary.

```
#defining the DL Model
from tensorflow.keras.models import Model
```

```
from tensorflow.keras.layers import Input, Embedding, LSTM, Bidirectional, D
from tensorflow.keras.optimizers import Adam

def seq2seq_model(article_vocab_size, highlight_vocab_size, embedding_dim, L
  #Encoder
  inputs = Input(shape=(max_article,), name='inputs')

  #encoder embedding
  #converts words to dense vector representations before passing to lstm
  embedding = Embedding(input_dim = article_vocab_size, output_dim = embeddi
                        input_length = max_article, name='embedding')(inputs

  #Bi-LSTM for Encoder
  bi_lstm = Bidirectional(LSTM(LSTM_units,
                               return_sequences=True, #outputs entire sequen
                               return_state=True, # returns the hidden state
                               dropout=dropout_rate, #reduces overfitting
                               name='bi_lstm_encoder'))
  encoder_outputs, forward_h, forward_c, backward_h, backward_c = bi_lstm(em
  #because Bi-LSTM captures data from both forward and backward direction fo

  #join / concatenate both states (forward + backword) for decoder
  states = [Concatenate()([forward_h, backward_h]),
            Concatenate()([forward_c, backward_c])]


  #Decoder
  inputs_decoder = Input(shape=(max_highlight,), name='inputs_decoder')

  #decoder embedding
  #converts words to dense vector representations
  embedding_decoder = Embedding(input_dim = highlight_vocab_size, output_dim
                                input_length = max_highlight, name='embeddin

  #Decoder LSTM (keeping states as initial states)
  lstm = LSTM(LSTM_units*2,  #double hidden units for better learning capaci
              return_sequences=True, #outputs entire sequence
              return_state=True, #return hidden and all other states
              dropout=dropout_rate, #reduces overfitting
              name='lstm')
  decoder_outputs, _, _ = lstm(embedding_decoder, initial_state=states)

  #Attention Mechanism, puts attention weights to most important words
  attention = Attention(name='attention')
  context_vector = attention([decoder_outputs, encoder_outputs])

  #join the context vector with the decoder output
  decoder_context = Concatenate(axis=-1, name='concat')([decoder_outputs, co

  #Dropout for Regularisation
  decoder_context = Dropout(dropout_rate)(decoder_context)

  #Output Layer
  #Softmax activation function here, ensures that the model outputs the word
  decoder_outputs = Dense(highlight_vocab_size, activation='softmax', name='
```

```
  #Model
  model = Model(inputs=[inputs, inputs_decoder], outputs=decoder_outputs)

  model.compile(optimizer=Adam(learning_rate=0.0005),
                loss='sparse_categorical_crossentropy',
                metrics=['accuracy'])
  return model

#build Model
model = seq2seq_model(
    article_vocab_size=article_vocab_size,
    highlight_vocab_size=highlight_vocab_size,
    embedding_dim=embedding_dim,
    LSTM_units=LSTM_units,
    max_article=max_article,
    max_highlight=max_highlight
)

model.summary()
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/embedding.py:90
  warnings.warn(
```

Model: "functional_7"

Layer (type)	Output Shape	Param #	Connected to
inputs (InputLayer)	(None, 200)	0	-
embedding (Embedding)	(None, 200, 32)	160,000	inputs[0][0]
inputs_decoder (InputLayer)	(None, 40)	0	-
bidirectional_7 (Bidirectional)	[(None, 200, 64), (None, 32), (None, 32), (None, 32), (None, 32)]	16,640	embedding[0][0]
embedding_decoder (Embedding)	(None, 40, 32)	160,000	inputs_decoder[0…
concatenate_14 (Concatenate)	(None, 64)	0	bidirectional_7[… bidirectional_7[…
concatenate_15 (Concatenate)	(None, 64)	0	bidirectional_7[… bidirectional_7[…
lstm (LSTM)	[(None, 40, 64), (None, 64), (None, 64)]	24,832	embedding_decode… concatenate_14[0… concatenate_15[0…
attention (Attention)	(None, 40, 64)	0	lstm[0][0], bidirectional_7[…
concat (Concatenate)	(None, 40, 128)	0	lstm[0][0], attention[0][0]
dropout_7 (Dropout)	(None, 40, 128)	0	concat[0][0]
outputs (Dense)	(None, 40, 5000)	645,000	dropout_7[0][0]

Training and Fine-Tuning

```
#Define callbacks
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

callbacks = [
    EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)
    ModelCheckpoint('seq2seq_model.keras', monitor='val_loss', save_best_onl
]
```

```
#Reshape the target sequences to have dimensions for sparse categorical cros
# Without reshaping, the loss function will try to compare (batch_size, leng
#with my targets (batch_size, length),which would give dimension mismatch er
train_highlight_target = train_highlight_pad.reshape(train_highlight_pad.sha
val_highlight_target = val_highlight_pad.reshape(val_highlight_pad.shape[0],
```

Implementing training with a few different experiments for better performance and finding the best model

```
!pip install rouge_score
```

```
Requirement already satisfied: rouge_score in /usr/local/lib/python3.11/dist-
Requirement already satisfied: absl-py in /usr/local/lib/python3.11/dist-pack
Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-package
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packag
Requirement already satisfied: six>=1.14.0 in /usr/local/lib/python3.11/dist-
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packag
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packa
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.11/d
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-package
```

```
from tensorflow.keras.optimizers import Adam, RMSprop, SGD, Adagrad
from rouge_score import rouge_scorer
import numpy as np
import datetime
import time

# Define a function to evaluate ROUGE scores
def compute_rouge_scores(predictions, references):
    scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_st

    # Convert predictions and references to strings before scoring
    scores = []
    for ref, pred in zip(references, predictions):
        # Flatten and convert to strings
        ref_str = " ".join(map(str, ref.flatten()))
        pred_str = " ".join(map(str, pred.flatten()))

        scores.append(scorer.score(ref_str, pred_str))

    avg_scores = {
        'rouge1': np.mean([score['rouge1'].fmeasure for score in scores]),
        'rouge2': np.mean([score['rouge2'].fmeasure for score in scores]),
        'rougeL': np.mean([score['rougeL'].fmeasure for score in scores])
    }

    return avg_scores

from tensorflow.keras.callbacks import ModelCheckpoint
```

```
# run_experiment function to include optimizers
def run_experiment(params, train_data, val_data, test_data, experiment_name)
    print(f"\n\n{'='*50}")
    print(f"Starting experiment: {experiment_name}")
    print(f"Parameters: {params}")
    print(f"{'='*50}\n")

    # Extracting parameters from previous code
    article_vocab_size = params.pop('article_vocab_size')
    highlight_vocab_size = params.pop('highlight_vocab_size')
    max_article = params.pop('max_article')
    max_highlight = params.pop('max_highlight')
    epochs = params.pop('epochs')
    batch_size = params.pop('batch_size')
    learning_rate = params.pop('learning_rate')
    optimizer_name = params.pop('optimizer')

    # Selecting optimizer based on the experiment configuration
    if optimizer_name == 'adam':
        optimizer = Adam(learning_rate=learning_rate)
    elif optimizer_name == 'sgd':
        optimizer = SGD(learning_rate=learning_rate)
    elif optimizer_name == 'adagrad':
        optimizer = Adagrad(learning_rate=learning_rate)

    # Extracting the data
    train_article_pad, train_decoder_input, train_highlight_target = train_d
    val_article_pad, val_decoder_input, val_highlight_target = val_data
    test_article_pad, test_decoder_input, test_highlight_target = test_data

    # Recording start time
    start_time = time.time()

    # Model creation
    def create_model(params, optimizer):
    # Encoder
      inputs = Input(shape=(params['max_article'],), name='inputs')

      # encoder embedding
      embedding = Embedding(input_dim=params['article_vocab_size'], output_d
                            input_length=params['max_article'], name='embe

      # Bi-LSTM for Encoder
      dropout_rate = params.get('dropout', 0.2)
      bi_lstm = Bidirectional(LSTM(params['LSTM_units'],
                                   return_sequences=True,
                                   return_state=True,
                                   dropout=dropout_rate,
                                   name='bi_lstm_encoder'))
      encoder_outputs, forward_h, forward_c, backward_h, backward_c = bi_lst

      # join / concatenate both states (forward + backward)
      states = [Concatenate()([forward_h, backward_h]),
                Concatenate()([forward_c, backward_c])]
```

```
    # Decoder
    inputs_decoder = Input(shape=(params['max_highlight'],), name='inputs_

    # decoder embedding
    embedding_decoder = Embedding(input_dim=params['highlight_vocab_size']
                                  input_length=params['max_highlight'],

    # Decoder LSTM
    lstm = LSTM(params['LSTM_units']*2,
                return_sequences=True,
                return_state=True,
                dropout=dropout_rate,
                name='lstm')
    decoder_outputs, _, _ = lstm(embedding_decoder, initial_state=states)

    # Attention Mechanism
    attention = Attention(name='attention')
    context_vector = attention([decoder_outputs, encoder_outputs])

    # join the context vector with the decoder output
    decoder_context = Concatenate(axis=-1, name='concat')([decoder_outputs

    # Dropout for Regularization
    decoder_context = Dropout(dropout_rate)(decoder_context)

    # Output Layer
    decoder_outputs = Dense(params['highlight_vocab_size'], activation='so

    # Model
    model = Model(inputs=[inputs, inputs_decoder], outputs=decoder_outputs

    model.compile(optimizer=optimizer,
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    return model

    # Defining the ModelCheckpoint callback to save the best model in Save
  checkpoint_path = f"best_model_{experiment_name}.keras"
  callbacks = [
      EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=T
      ModelCheckpoint(checkpoint_path, monitor='val_loss', save_best_only=
  ]

  # Training model
  history = model.fit(
      [train_article_pad, train_decoder_input], train_highlight_target,
      validation_data=([val_article_pad, val_decoder_input], val_highlight
      epochs=epochs,
      batch_size=batch_size,
      callbacks=callbacks
  )

  # Calculate training time
  training_time = time.time() - start_time
```

```
    # Evaluating model on the test data
    test_predictions = model.predict([test_article_pad, test_decoder_input])
    test_predictions = np.argmax(test_predictions, axis=-1)

    # Computing ROUGE scores for the predictions
    references = test_highlight_target  # Actual highlights from the test se
    rouge_scores = compute_rouge_scores(test_predictions, references)

    # Extracting results
    results = {
        'experiment_name': experiment_name,
        'optimizer': optimizer_name,
        'embedding_dim': params['embedding_dim'],
        'LSTM_units': params['LSTM_units'],
        'dropout': params.get('dropout', 0.2),
        'batch_size': batch_size,
        'learning_rate': learning_rate,
        'epochs_completed': len(history.history['loss']),
        'final_train_loss': history.history['loss'][-1],
        'final_train_acc': history.history['accuracy'][-1],
        'final_val_loss': history.history['val_loss'][-1],
        'final_val_acc': history.history['val_accuracy'][-1],
        'rouge1': rouge_scores['rouge1'],
        'rouge2': rouge_scores['rouge2'],
        'rougeL': rouge_scores['rougeL'],
        'training_time_seconds': training_time
    }

    print(f"\nExperiment completed in {training_time:.2f} seconds")
    print(f"Final results: Train Loss: {results['final_train_loss']:.4f}, Tr
    print(f"             Val Loss: {results['final_val_loss']:.4f}, Val Acc:
    print(f"             ROUGE-1: {results['rouge1']:.4f}, ROUGE-2: {results

    return results, model, history

# the run_experiments function
def run_experiments():
    # Preparing data
    train_highlight_target = train_highlight_pad.reshape(train_highlight_pad
    val_highlight_target = val_highlight_pad.reshape(val_highlight_pad.shape
    test_highlight_target = test_highlight_pad.reshape(test_highlight_pad.sh

    train_data = (train_article_pad, train_decoder_input, train_highlight_ta
    val_data = (val_article_pad, val_decoder_input, val_highlight_target)
    test_data = (test_article_pad, test_decoder_input, test_highlight_target

    # Configurations for experiments with different optimizers
    experiment_configs = [
        {
            'experiment_name': 'adam_optimizer',
            'article_vocab_size': article_vocab_size,
            'highlight_vocab_size': highlight_vocab_size,
            'embedding_dim': 64,
            'LSTM_units': 64,
            'dropout': 0.3,
```

```
        'batch_size': 32,
        'max_article': max_article,
        'max_highlight': max_highlight,
        'epochs': 10,
        'learning_rate': 0.001,
        'optimizer': 'adam'
    },

    {
        'experiment_name': 'sgd_optimizer',
        'article_vocab_size': article_vocab_size,
        'highlight_vocab_size': highlight_vocab_size,
        'embedding_dim': 64,
        'LSTM_units': 64,
        'dropout': 0.3,
        'batch_size': 32,
        'max_article': max_article,
        'max_highlight': max_highlight,
        'epochs': 10,
        'learning_rate': 0.001,
        'optimizer': 'sgd'
    },
    {
        'experiment_name': 'adagrad_optimizer',
        'article_vocab_size': article_vocab_size,
        'highlight_vocab_size': highlight_vocab_size,
        'embedding_dim': 64,
        'LSTM_units': 64,
        'dropout': 0.3,
        'batch_size': 32,
        'max_article': max_article,
        'max_highlight': max_highlight,
        'epochs': 10,
        'learning_rate': 0.001,
        'optimizer': 'adagrad'
    }
]

# Storing results
all_results = []

# Creating timestamp for saving results
timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")

# Running experiments
for config in experiment_configs:
    experiment_name = config['experiment_name']
    results, model, history = run_experiment(config.copy(), train_data,
    all_results.append(results)

    # Save the model and other outputs as before

# Final results
final_df = pd.DataFrame(all_results)
print("\n\nFinal Results:")
```

```
    print(final_df[['experiment_name', 'final_train_loss', 'final_val_loss',

    return final_df

# Running all experiments
results_df = run_experiments()
```

```
==================================================
Starting experiment: adam_optimizer
Parameters: {'experiment_name': 'adam_optimizer', 'article_vocab_size': 500
==================================================

Epoch 1/10
31/32 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.1873 - loss: 8.4883
Epoch 1: val_loss improved from inf to 8.03362, saving model to best_model_a
32/32 ━━━━━━━━━━━━━━━━━━━━ 5s 61ms/step - accuracy: 0.1905 - loss: 8.4838 -
Epoch 2/10
30/32 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.2713 - loss: 7.4676
Epoch 2: val_loss improved from 8.03362 to 6.32271, saving model to best_mod
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.2709 - loss: 7.4245 -
Epoch 3/10
31/32 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.2642 - loss: 5.9326
Epoch 3: val_loss improved from 6.32271 to 6.04166, saving model to best_mod
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.2646 - loss: 5.9274 -
Epoch 4/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.2834 - loss: 5.7045
Epoch 4: val_loss improved from 6.04166 to 5.91309, saving model to best_mod
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.2837 - loss: 5.7028 -
Epoch 5/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.3085 - loss: 5.5660
Epoch 5: val_loss improved from 5.91309 to 5.83354, saving model to best_mod
32/32 ━━━━━━━━━━━━━━━━━━━━ 2s 37ms/step - accuracy: 0.3088 - loss: 5.5644 -
Epoch 6/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.3352 - loss: 5.4276
Epoch 6: val_loss improved from 5.83354 to 5.79949, saving model to best_mod
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3352 - loss: 5.4281 -
Epoch 7/10
30/32 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.3361 - loss: 5.4326
Epoch 7: val_loss improved from 5.79949 to 5.78953, saving model to best_mod
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.3366 - loss: 5.4305 -
Epoch 8/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 0.3458 - loss: 5.3487
Epoch 8: val_loss improved from 5.78953 to 5.77777, saving model to best_mod
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - accuracy: 0.3457 - loss: 5.3499 -
Epoch 9/10
30/32 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 0.3554 - loss: 5.2656
Epoch 9: val_loss improved from 5.77777 to 5.77599, saving model to best_mod
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3542 - loss: 5.2755 -
Epoch 10/10
31/32 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.3452 - loss: 5.3392
Epoch 10: val_loss improved from 5.77599 to 5.77591, saving model to best_mo
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 39ms/step - accuracy: 0.3452 - loss: 5.3403 -
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 86ms/step

Experiment completed in 17.89 seconds
Final results: Train Loss: 5.3581, Train Acc: 0.3446
```

```
                  Val Loss: 5.7759, Val Acc: 0.3127
                  ROUGE-1: 0.4059, ROUGE-2: 0.2653, ROUGE-L: 0.4059


    ==================================================
    Starting experiment: sgd optimizer
```

In conclusion,

due to time and resources constraint, I could only perform these many combinations of fine tuning, as it was taking a long time to learn. According to a research made in 2021 by a team of people on the same process (Bi-LSTM + Attention), it took a minimum of 40 epochs to train and achieve the accuracy of 93.6%.

Here, In my approach, after experimenting with a few hyperparameters (Optimizers), and training for 10 epochs, I got comparatively very less accuracy and rougue scores, specifically, the best one was with Adam Optimizer with rouge scores of **40.58**, **26.52** and **40.58** respectively and accuracy of **34.46%**, which with more samples, epochs and training would result in much higher scores and accuracy. Another approach for the Summarization task for a better Text Summarizer, I decided to fine-tune a pre-trained model called **BART** (Bidirectional AutoRegressive Transformer) by *facebook* to train the **CNN/DailyMail Dataset from the hugging face library**.

For Real-world use of the model, I have then **deployed the app on Hugging Face Spaces using Gradio**.

> *The steps to access the link to my web app are described at the end of the colab file.*

Using Bidirectional AutoRegressive Transformer (BART)

Here, I am building a Text Summarizer using BART (Bidirectional Autoregressive Transformer) which will help to generate a short summary of the long time consuming documents.

The whole process is divided into **2 sections**:

data loading, pre-processing and training loading model and launching web app via Gradio. let us first install the transformer and datasets because I am using the version 3.0.0 of the cnn/dailymails dataset from the hugging face library.

Data Loading, Pre-processing and Training

```
!pip install transformers datasets
```

The next step, I have imported the necessary libraries as follows:

- BartTokenizer - it will tokenize the text and feed into the model
- BartForConditionalGeneration - this is a BART model class for doing tasks like summarization
- TrainingArguments, Trainer - these are used for fine-tuning and training the model.
- DataCollatorForSeq2Seq - this will help prepare correct batch sizes for the task for faster training.

I have then, mounted my drive and saved the fine-tuned model and tokenizer there for future use.

```
from transformers import BartTokenizer, BartForConditionalGeneration, Traini
from datasets import load_dataset
import os
from google.colab import drive

# Mount Google Drive to save your model persistently
drive.mount('/content/drive')

# Create a directory in your Google Drive to save the model
save_directory = '/content/drive/MyDrive/summarizer_model'
os.makedirs(save_directory, exist_ok=True)
```

The below code is loading the pre-trained BART model.

```
# Load dataset
dataset = load_dataset('cnn_dailymail', '3.0.0')
model_name = 'facebook/bart-large-cnn'
tokenizer = BartTokenizer.from_pretrained(model_name)
model = BartForConditionalGeneration.from_pretrained(model_name)
```

```
# Define preprocessing function
'''
this code will take article and highlight text as input and target and set a
truncation = true refers to cutting the longer sentences and padding means a
to shorter sentences for equal length.
'''
def preprocess(example):
  inputs = tokenizer(example['article'], max_length=512, truncation=True, pa
  targets = tokenizer(example['highlights'], max_length=128, truncation=True
```

```
    inputs['labels'] = targets['input_ids'] # (assigns token ids of highlights
    return inputs

# Preprocess and prepare data
tokenized_dataset = dataset.map(preprocess, batched=True, remove_columns=dat
train_dataset = tokenized_dataset['train'].select(range(1000))
val_dataset = tokenized_dataset['validation'].select(range(200))
```

```
# Set up training arguments
training_args = TrainingArguments(
    output_dir=save_directory + '/checkpoints',
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=8, #accumulates gradients over 8 steps befor
    evaluation_strategy='epoch',
    save_strategy='epoch',
    num_train_epochs=1,
    learning_rate=3e-5,
    fp16=True, #with this the training is faster and less memory-intensive o
    report_to='none', #does not report to external platforms like tensorboar
    logging_dir=save_directory + '/logs',
```

```
# Set up trainer and train
'''
Collator automatically pads sequences, assigns weight (attention) and assemb
'''
from transformers import DataCollatorForSeq2Seq
data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=model)
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    tokenizer=tokenizer,
    data_collator=data_collator,
)

trainer.train()

# Save the fine-tuned model to Google Drive
model_save_path = save_directory + '/bart-summarizer-cnn'
model.save_pretrained(model_save_path)
tokenizer.save_pretrained(model_save_path)
print(f"Model saved to {model_save_path}")

# Test saving/loading worked by loading and testing
test_model = BartForConditionalGeneration.from_pretrained(model_save_path)
test_tokenizer = BartTokenizer.from_pretrained(model_save_path)
print("Successfully loaded saved model and tokenizer!")
```

Loading model and Launching Gradio

When you want access the public URL for the deployed model, you only have to run the codes in this section which loads the fine-tuned model saved on my drive and launches the app.

```
!pip install gradio
```

```
Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/lib/python3
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: brotli>=1.1.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: fastapi<1.0,>=0.115.2 in /usr/local/lib/python
Requirement already satisfied: ffmpy in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: gradio-client==1.14.0 in /usr/local/lib/python
Requirement already satisfied: groovy~=0.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: httpx<1.0,>=0.24.1 in /usr/local/lib/python3.1
Requirement already satisfied: huggingface-hub<2.0,>=0.33.5 in /usr/local/lib
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/lib/python3
Requirement already satisfied: numpy<3.0,>=1.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/python3.12/
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib/python3.12
Requirement already satisfied: pydantic<=2.12.3,>=2.0 in /usr/local/lib/pytho
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/lib/pyt
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.12/
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/local/lib/pyth
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/python
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/local/lib/pytho
Requirement already satisfied: tomlkit<0.14.0,>=0.12.0 in /usr/local/lib/pyth
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.12/
Requirement already satisfied: typing-extensions~=4.0 in /usr/local/lib/pytho
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: websockets<16.0,>=13.0 in /usr/local/lib/pytho
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dis
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/pytho
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/di
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/pytho
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/pyt
Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.1
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python
```

```
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/pyth
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pyt
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.1
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-p
```

In the code below, I first load my fine-tuned model from my drive and for security concerns, check if it available in the drive. If under any scenario, the model does not exist, it will continue using the pre-trained 'facebook/bart-large-cnn' model instead.

```
from transformers import BartTokenizer, BartForConditionalGeneration
import gradio as gr
from google.colab import drive
import os

# Mount Google Drive
drive.mount('/content/drive')

# Path to your saved model
model_path = '/content/drive/MyDrive/summarizer_model/bart-summarizer-cnn'

# Check if model exists
if os.path.exists(model_path):
    print(f"Loading model from {model_path}")
    model = BartForConditionalGeneration.from_pretrained(model_path)
    tokenizer = BartTokenizer.from_pretrained(model_path)
    print("Model loaded successfully!")
else:
    print(f"Model not found at {model_path}")
    print("Loading pre-trained model instead...")
    model = BartForConditionalGeneration.from_pretrained('facebook/bart-larg
    tokenizer = BartTokenizer.from_pretrained('facebook/bart-large-cnn')
```

```
Mounted at /content/drive
Loading model from /content/drive/MyDrive/summarizer_model/bart-summarizer-cn
/usr/local/lib/python3.12/dist-packages/transformers/models/bart/configuratio
  warnings.warn(
---------------------------------------------------------------------------
OSError                                   Traceback (most recent call last)
/tmp/ipython-input-600108248.py in <cell line: 0>()
     13 if os.path.exists(model_path):
     14     print(f"Loading model from {model_path}")
---> 15     model = BartForConditionalGeneration.from_pretrained(model_path)
     16     tokenizer = BartTokenizer.from_pretrained(model_path)
     17     print("Model loaded successfully!")

                              2 frames
/usr/local/lib/python3.12/dist-packages/transformers/modeling_utils.py in
_get_resolved_checkpoint_files(pretrained_model_name_or_path, subfolder,
variant, gguf_file, from_tf, from_flax, use_safetensors, cache_dir,
force_download, proxies, local_files_only, token, user_agent, revision,
commit_hash, is_remote_code, transformers_explicit_filename)
    987                 )
    988             else:
--> 989                 raise OSError(
    990                     f"Error no file named
{_add_variant(WEIGHTS_NAME, variant)}, {_add_variant(SAFE_WEIGHTS_NAME,
variant)},"
    991                     f" {TF2_WEIGHTS_NAME}, {TF_WEIGHTS_NAME +
'.index'} or {FLAX_WEIGHTS_NAME} found in directory"

OSError: Error no file named pytorch_model.bin, model.safetensors
```

```
# Define the summarization function to use the fine-tuned BART model to ger
'''
return_tensors=pt means that the output will be a PyTorch tensor which is t
num_beams = 4, means the model will look for top 4 most likely next words
in the sequence based on overall probability.
length penalty will tell the model to not generate very large summary and i
'''
def summarize_article(article, creativity):
    inputs = tokenizer(article, max_length=512, return_tensors='pt', trunca
    summary_ids = model.generate(
        inputs['input_ids'],
        max_length=160,
        min_length=60,
        do_sample=True,
        temperature=creativity,
        top_k=50,
        top_p=0.95,
        num_beams=4,
        repetition_penalty=1.2,
        early_stopping=True
    )
```

```
    '''
    ids[0] will take the first generated summary out of 4 beam results
    skip_special_tokens will skip the padding tokens from the final summary
    '''
```

```
# Create and launch Gradio interface
'''
gr.interface() will create a gradio interface object, which will have my mod
input and output, and title and description for the interface.
when you run interface.launch, colab will generate a public URL to access th
'''
interface = gr.Interface(
    fn=summarize_article,
    inputs=[gr.Textbox(label="Enter Article"),
            gr.Slider(0.5,1.2, value=0.9, label="Creativity Level")],
    outputs='text',
    title='Abstractive Article Summarizer',
    description='Enter an article to get a summary and adjust creativity to
)

interface.launch()
```

```
It looks like you are running Gradio on a hosted a Jupyter notebook. For the

Colab notebook detected. To show errors in colab notebook, set debug=True in
* Running on public URL: https://b760e23a6e82668a59.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrade
```

gradio

No interface is running right now